SIMATS ENGINEERING

CO1-CONSTRAINT BASED PROBLEM SOLVING

NAME:Shalini Douglas

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

Question 4. Map Colouring of South Indian States (CSP with Heuristics)

1. Problem understanding: CSP formulation

Goal: assign each state one colour from {Red, Green, Blue} so every neighbouring pair has different colours.

Variables: TN, KL, KA, AP, TG, GA, MH.

Domains: D(TN)=D(KL)=D(KA)=D(AP)=D(TG)=D(GA)=D(MH)={Red, Green, Blue}.

Binary constraints: X ≠ Y for each listed neighbouring pair: TN–KL, TN–KA, TN–AP, KL–KA, KA–AP, KA–TG, KA–GA, KA–MH, AP–TG, GA–MH, MH–TG.

2. Constraint graph and variable ordering

Constraint graph (an edge means the two states must have different colours):

- TN: KL, KA, AP
- KL: TN, KA
- KA: TN, KL, AP, TG, GA, MH
- AP: TN, KA, TG
- TG: KA, AP, MH
- GA: KA, MH
- MH: KA, GA, TG

Degree heuristic: KA has the highest degree (6 neighbours), so choose KA first. Assign KA = Red. After forward checking, each of KL, TN, AP, TG, GA and MH loses Red and has domain {Green, Blue}.

MRV for the remaining variables: all unassigned variables initially have two values after KA=Red, so MRV is tied. Break the tie by choosing a variable with the most unassigned neighbours (degree tie-break); choose AP, then TN, then KL, then TG, then MH, then GA. (Other tie-break orders may also work.)

3. Backtracking with forward checking: domains after each assignment

Use the stated tie-break order. Domains shown are the current domains of unassigned variables after forward checking; assigned variables are shown separately.

Step	Domains after assignment / forward checking
Start	All variables {R,G,B}
KA = Red	TN, KL, AP, TG, GA, MH: {G,B}
AP = Green	TN: {B}; KL: {G,B}; TG: {B}; GA: {G,B}; MH: {G,B}
TN = Blue	KL: {G}; TG: {B}; GA: {G,B}; MH: {G,B}
KL = Green	TG: {B}; GA: {G,B}; MH: {G,B}
TG = Blue	GA: {G,B}; MH: {G} (TG–MH constraint removes Blue)
MH = Green	GA: {B} (GA–MH constraint removes Green)
GA = Blue	Complete; no domain wipeout

Note: R=Red, G=Green, B=Blue. Each chosen value is checked against all already assigned neighbours before continuing. No backtrack is required with this ordering.

4. Final colouring and heuristic justification

Final assignment: KA=Red, AP=Green, TN=Blue, KL=Green, TG=Blue, MH=Green, GA=Blue.

Every listed edge joins different colours: TN differs from KL, KA and AP; KL differs from KA; KA differs from AP, TG, GA and MH; AP differs from TG; GA differs from MH; and MH differs from TG.

The degree heuristic starts with KA because it constrains the most other variables. Forward checking immediately removes Red from all its neighbours. MRV then prioritizes variables with the fewest legal values; tie-breaking makes the order reproducible. These choices expose restrictions early and avoid exploring many doomed partial assignments.

Question 5. 4×4 Sudoku (Constraint Propagation with AC-3)

1. Important note about the supplied puzzle

The puzzle text appears to have lost a blank/cell separator and, read literally as rows "– 2 – 3 / – – – – / – 2 4 4 / – – – –", it is inconsistent: row 3 contains two 4s, and column 2 repeats 2. No valid Sudoku solution exists for that literal grid. To demonstrate the requested CSP/AC-3 method, the worked solution below uses an explicitly corrected, valid example puzzle. Please compare it with the original question image if exact givens are required.

Worked example used (– means blank):

–	2	–	3
3	–	1	–
–	1	3	–
4	–	–	1

2. Variables, domains and constraints

Variables: Xrc for each cell in row r and column c, where r,c ∈ {1,2,3,4}.

Domain: each blank cell initially has values from {1,2,3,4}; remove any value that conflicts with a given digit in the same row, column or 2×2 box.

Constraints: Alldiff(Xr1, Xr2, Xr3, Xr4) for every row; Alldiff(X1c, X2c, X3c, X4c) for every column; and Alldiff over the four cells in each 2×2 box.

3. Initial domains after removing conflicts with givens

Blank cell	Initial domain
R1C1	{1}
R1C3	{4}
R2C2	{4}
R2C4	{2,4}
R3C1	{2}
R3C4	{2,4}
R4C2	{3}
R4C3	{2}

Givens in the worked example are R1C2=2, R1C4=3, R2C1=3, R2C3=1, R3C2=1, R3C3=3, R4C1=4 and R4C4=1.

4. AC-3 constraint propagation

AC-3 repeatedly revises arcs between cells that share a row, column or box. For a binary inequality arc (X,Y), a value in D(X) is retained only if D(Y) contains at least one different value. With Alldiff, singleton domains also remove their value from other cells in the same unit.

1. R1C3 is forced to 4 by its row/column/box restrictions. Remove 4 from peers in its row, column and box.
2. R2C2 is forced to 4; remove 4 from its row, column and box peers.
3. R3C1 is forced to 2; remove 2 from peers in its row, column and box.
4. R4C2 is forced to 3; remove 3 from peers in its row, column and box.
5. R4C3 is forced to 2; remove 2 from peers in its row, column and box.
6. The remaining cells become singletons through repeated row/column/box propagation: R1C1=1, R2C4=2, R3C4=4.

For the corrected example, propagation yields the complete grid below without needing search. AC-3 is usually described for binary constraints; the Alldiff propagation above is the equivalent practical use of singleton elimination on each all-different unit.

5. Completed grid

1	2	4	3
3	4	1	2
2	1	3	4
4	3	2	1

Each row, column and 2×2 box contains 1, 2, 3 and 4 exactly once. If the original givens differ from the corrected example, the final grid and domain trace must be recalculated.

Question 8. 4-Queens (Local Search with Min-Conflicts)

1. CSP formulation

Variables: Q1, Q2, Q3, Q4, where Qc is the row occupied by the queen in column c.

Domains: D(Qc)={1,2,3,4} for each column c.

Constraints for every pair of columns i≠j: Qi ≠ Qj (no shared row) and |Qi−Qj| ≠ |i−j| (no shared diagonal). Exactly one queen is in each column by representation.

2. Initial board and conflict counts

Initial rows by column: (1, 3, 4, 2). A conflict is an attacking pair of queens. Q2 and Q3 attack diagonally; all other pairs do not. Thus there is one conflicting pair. Per-queen counts count how many other queens attack that queen.

Q			
			Q
	Q		
		Q	

Initial per-queen conflict counts: Q1=0, Q2=1, Q3=1, Q4=0.

3. Min-Conflicts steps

Rule used: select the leftmost queen currently in conflict; move it to a different row with the fewest conflicts, breaking ties by lowest row number. Excluding the current row from candidate moves avoids a no-op when the current row ties for the minimum.

State / move	Rows (Q1,Q2,Q3,Q4)	Per-queen conflicts	Explanation
Initial	(1,3,4,2)	(0,1,1,0)	Q2 and Q3 form one diagonal conflict.
Move 1: Q2 → row 1	(1,1,4,2)	(1,1,0,0)	Q2 is leftmost in conflict. Rows 1 and 4 each give one conflict; row 1 is the lower row, so choose it.
Move 2: Q1 → row 3	(3,1,4,2)	(0,0,0,0)	Q1 is now the leftmost queen in conflict. Excluding its current row, rows 3 and 4 tie at one conflict; choose row 3. All conflicts are removed.

4. Final solution and justification

Final rows by column: (3, 1, 4, 2). The queens occupy (row, column) positions (3,1), (1,2), (4,3), and (2,4). No two queens share a row or diagonal, so this is a valid solution.

Min-Conflicts works well for large n-Queens instances because it starts from a complete assignment and repeatedly repairs a violated constraint rather than constructing a solution one variable at a time. Each step evaluates local conflicts and usually makes

rapid progress. It is a heuristic, so it is not guaranteed to avoid every local minimum; tie-breaking and occasional restarts are common practical safeguards.